
C 程序设计基础及实验

主讲教师: 纪守领

学院: 计算机科学与技术

NESA Lab: <https://nesa.zju.edu.cn/>

Email: sji@zju.edu.cn

Chapter 10 函数与程序结构

10.1 有序表的操作

10.2 汉诺塔问题

10.3 长度单位转换

10.4 大程序构成

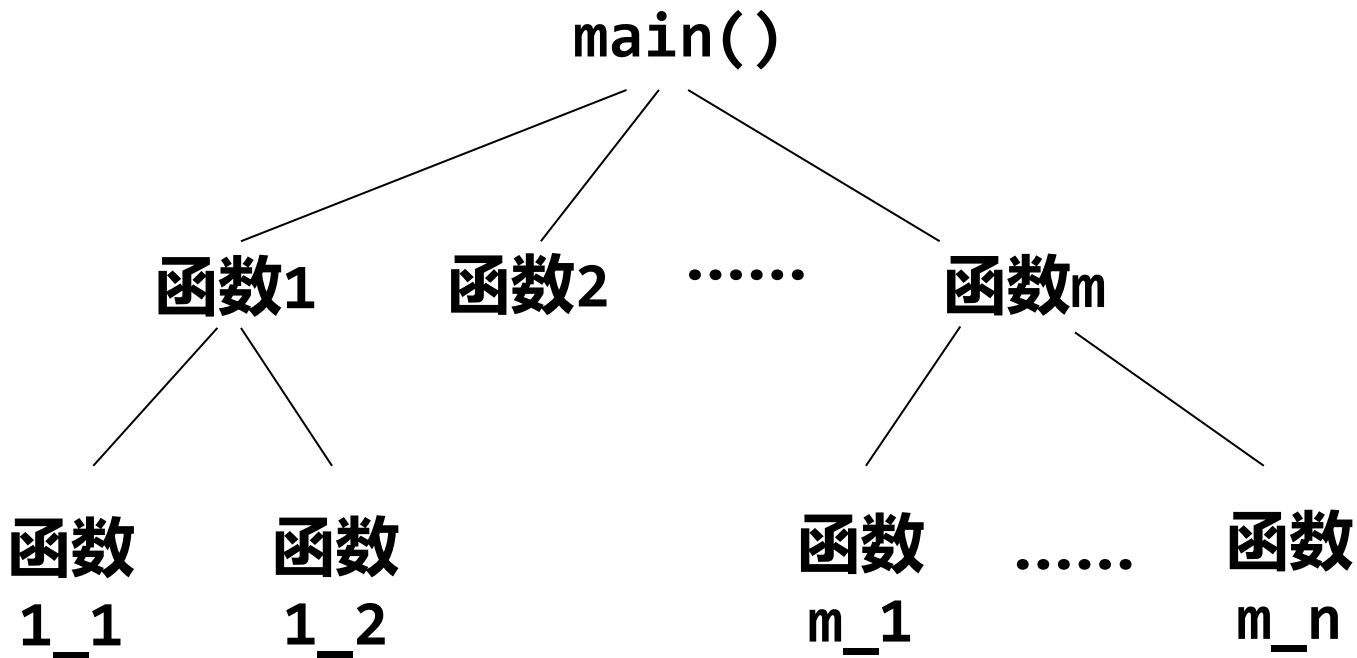
本章要点

- 怎样把多个函数组织起来？
- 怎样用结构化程序设计的思想解决问题？
- 怎样用函数嵌套求解复杂的问题？
- 怎样用函数递归解决问题？
- 如何使用宏？
- 如何使用多文件模块构建较大规模程序？

10.1 有序表的操作

- 使用结构化程序设计方法解决复杂的问题
 - 把大问题分解成若干小问题，小问题再进一步分解成若干更小的问题
 - 写程序时，用main()解决整个问题，它调用解决小问题的函数
 - 这些函数又进一步调用解决更小问题的函数，从而形成函数的嵌套调用

程序结构



10.1.1 程序解析-有序表操作

- 例10-1 首先输入一个无重复元素的、从小到大排列的有序表，并在屏幕上显示以下菜单（编号和选项），用户可以反复对该有序表进行插入、删除和查找操作，也可以选择结束。当用户输入编号1~3和相关参数时，将分别对该有序表进行插入、删除和查找操作，输入其他编号，则结束操作

[1] Insert

[2] Delete

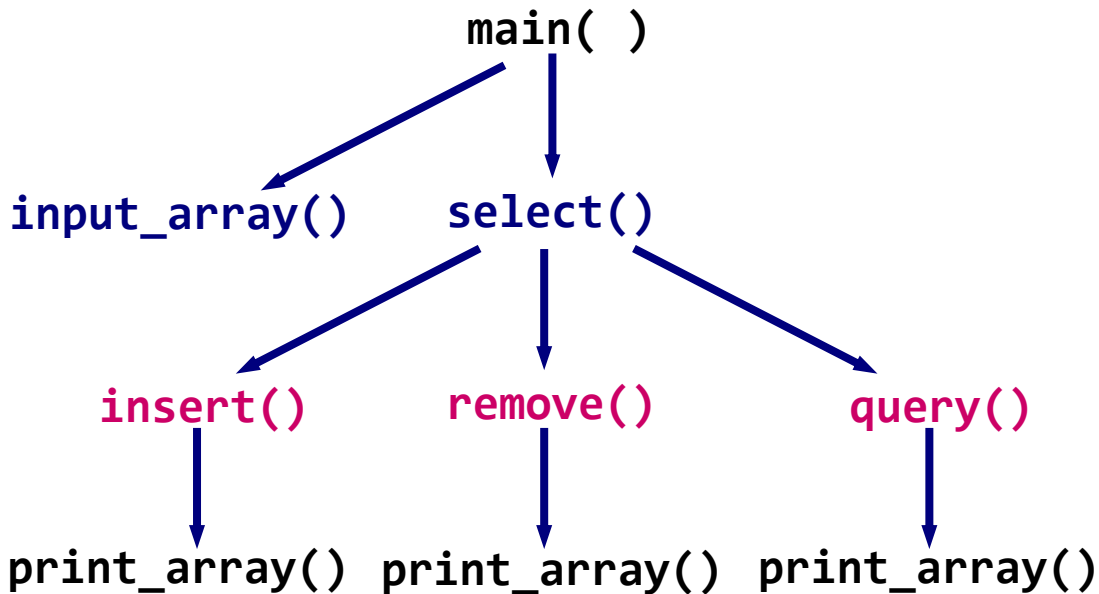
[3] Query

[Other option] End

例10-1 分析

- 输入有序表;
- 输入1、2、3选择插入、删除、查找操作, 其他输入结束;
 - 设计一个控制函数select(), 经它辨别用户输入的编号后, 调用相应的插入、删除和查找函数, 再调用有序表输出函数显示结果
- 分别设计函数实现有序表的插入、删除、查找和输入、输出等常规操作

例10-1 程序结构



4层结构，7个函数

降低程序的构思、编写、调试的复杂度，可读性好

例10-1 源程序

```
int Count = 0;
void select(int a[], int option, int value);
void input_array(int a[]);
void print_array(int a[]);
void insert(int a[], int value);
void remove(int a[], int value);
void query(int a[], int value);
int main(void)
{
    int option, value, a[100];
    input_array(a);          /* 调用函数输入有序数组 a */
    printf("[1] Insert\n");   printf("[2] Delete\n");
    printf("[3] Query\n");    printf("[Other option] End\n");
    while (1) {
        printf("Input option: ");    scanf("%d", &option);
        if (option < 1 || option > 3) {break;}
        printf("Input an element: "); scanf("%d", &value);
        select(a, option, value);    /* 调用控制函数 */
        printf("\n");
    }
    printf("Thanks.");
    return 0;
}
```

select()

```
/* 控制函数 */
void select(int a[], int option, int value)
{
    switch (option) {
        case 1:
            insert(a, value);    /* 调用插入函数在有序数组 a 中插入元素value */
            break;
        case 2:
            remove(a, value);    /* 调用删除函数在有序数组 a 中删除元素value */
            break;
        case 3:
            query(a, value);     /* 调用查询函数在有序数组 a 中查找元素value */
            break;
    }
}
```

input_array() / print_array()

```
/* 有序表输入函数 */
void input_array(int a[])
{   printf("Input the number of array elements: ");
    scanf("%d", &Count);
    printf("Input an ordered array element: ");
    for (int i = 0; i < Count; i ++) {
        scanf("%d", &a[i]);
    }
}

/* 有序表输出函数 */
void print_array(int a[])
{   printf("The ordered array a is: ");
    for (int i = 0; i < Count; i ++) { /* 相邻数字间空格分隔, 行末无空格 */
        if(i == 0){
            printf("%d", a[i]);
        }else{
            printf(" %d", a[i]);
        }
    }
}
```

insert()

```
/* 有序表插入函数 */
void insert(int a[ ], int value)
{   int i, j;
    for (i = 0; i < Count; i++) {           /* 定位：待插入的位置即退出循环时i的值 */
        if(value < a[i]){
            break;
        }
    }
    for (j = Count - 1; j >= i; j--){ /* 腾位：将a[i]~a[Count-1]向后顺移一位 */
        a[j+1] = a[j];
    }
    a[i] = value;                             /* 插入：将value 的值赋给a[i] */
    Count++;                                  /* 增1：数组a中待处理的元素数量增1 */
    print_array(a);                          /* 调用输出函数，输出插入后的有序数组a */
}
```

remove()

```
/* 有序表删除函数 */
void remove(int a[ ], int value)
{
    int i, index = -1;
    for(i = 0; i < Count; i++){
        if(value == a[i]){
            index = i;
            break;
        }
    }
    if(index == -1){
        printf("Failed to find the data, deletion failed.");
    }else{
        for(i = index; i < Count - 1; i++){
            a[i] = a[i+1];
        }
        Count--;
        print_array(a);
    }
}
```

/* index的值为-1表示没找到, 则表示找到 */
/* 定位: 找到则用index记录其下标 */
/* 没找到, 则输出相应的信息 */
/* 找到, 则删除a[index] */
/* 将a[Count-1]~ a[index+1]向前顺移一位 */
/* 减1: 数组a中待处理的元素数量减1 */
/* 调用输出函数, 输出删除后的有序数组a */

query()

```
/* 有序表二分法查询函数 */
void query(int a[ ], int value)
{   int mid, left = 0, right = Count - 1;           /* 开始时查找区间为整个数组 */
    while (left <= right){
        mid = (left + right) / 2;                    /* 得到中间位置 */
        if (value == a[mid]){                        /* 查找成功, 输出下标, 函数返回 */
            printf("The index is: %d", mid);
            return;
        }else if (value < a[mid]){                   /* 缩小查找区间为前半段, right前移 */
            right = mid - 1;
        }else{                                       /* 缩小查找区间为后半段, left后移 */
            left = mid + 1;
        }
    }
    printf("This element does not exist.");          /* value不在数组a中 */
}
```

10.1.2 函数的嵌套调用

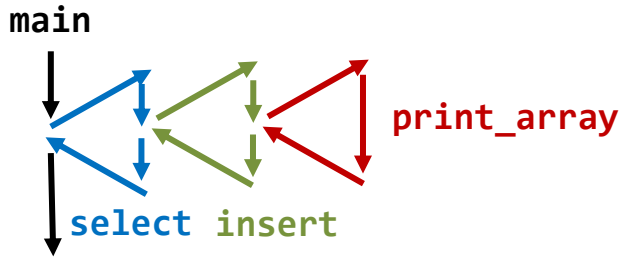
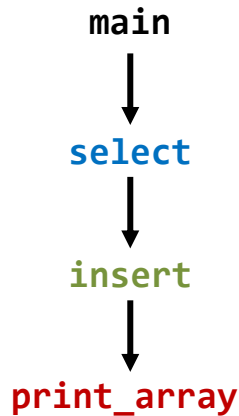
□ 嵌套调用

```
int main(void)
{
    .....
    select(a, option, value);
    .....
}

void select(int a[ ], int option, int value)
{
    .....
    insert(a, value);
    .....
}

void insert(int a[ ], int value)
{
    .....
    print_array(a);
    .....
}

void print_array(int a[ ])
{
    .....
}
```



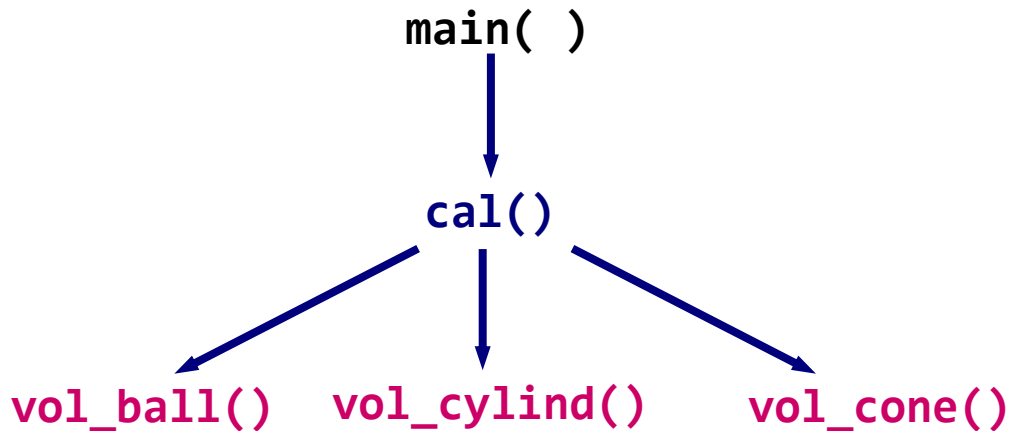
10.1.2 函数的嵌套调用

- 在一个函数中再调用其它函数的情况称为函数的嵌套调用
 - 如果函数A调用函数B，函数B再调用函数C，一个调用一个地嵌套下去，就构成了函数的嵌套调用
 - 具有嵌套调用函数的程序，需要分别定义多个不同的函数体，每个函数体完成不同的功能，它们合起来解决复杂的问题

程序解析-计算常用圆形体体积

- **例10-1** 设计一个常用圆形体体积计算器，采用命令方式输入1、2、3，分别选择计算球体、圆柱体、圆锥体的体积，并输入计算所需相应参数
- **分析：**
 - 输入1、2、3选择计算3种体积，其它输入结束计算
 - 设计一个控制函数cal()，经它辨别圆形体的类型，再调用计算球体、圆柱体、圆锥体体积的函数
 - 设计单独的函数计算不同圆形体的体积

程序结构



- 3层结构, 5个函数
- 降低程序的构思、编写、调试的复杂度
- 可读性好

源程序

```
#define PI 3.141592654
void cal ( int sel );
int main(void)
{
    int sel;
    while(1){                /* 永久循环, 通过循环体中 break 语句结束循环 */
        printf(" 1-计算球体体积\n");
        printf(" 2-计算圆柱体积\n");
        printf(" 3-计算圆锥体积\n");
        printf(" 其他-退出程序运行\n");
        printf("请输入计算命令: ");
        scanf("%d", &sel);
        if (sel < 1 || sel > 3)                /* 输入非1~3的数字, 循环结束 */
            break;
        else                                  /* 输入1~3, 调用cal() */
            cal(sel);
    }
    return 0;
}
```

源程序

```
double  vol_ball(void);           /* 函数声明 */
double  vol_cylind(void);
double  vol_cone(void );
/* 常用圆形体体积计算器的主控函数 */
void cal ( int sel )
{
    switch (sel) {
        case 1:                    /* 计算球体积 */
            printf("球体积为: %.2f\n",vol_ball());
            break;
        case 2:                    /* 计算圆柱体积 */
            printf("圆柱体积为: %.2f\n",vol_cylind( ));
            break;
        case 3:                    /* 计算圆锥体积 */
            printf("圆锥体积为: %.2f\n",vol_cone( ));
            break;
    }
}
```

源程序

```
double vol_ball(void);           /* 函数声明 */
double vol_cylind(void);
double vol_cone(void );
/* 常用圆形体体积计算器的主控函数 */
void cal ( int sel )
{
    switch (sel) {
        case 1:                  /* 计算球体积 */
            printf("球体积为: %.2f\n",vol_ball());
            break;
        case 2:                  /* 计算圆柱体积 */
            printf("圆柱体积为: %.2f\n",vol_cylind( ));
            break;
        case 3:                  /* 计算圆锥体积 */
            printf("圆锥体积为: %.2f\n",vol_cone( ));
            break;
    }
}
```

```
/* 计算球体体积  $V=4/3*PI*r*r*r$  */
double vol_ball( )
{
    double r;

    printf("请输入球的半径: ");
    scanf("%lf", &r);
    return(4.0/3.0*PI*r*r*r);
}
```

源程序

```
double vol_ball(void);           /* 函数声明 */
double vol_cylind(void);
double vol_cone(void );
/* 常用圆形体体积计算器的主控函数 */
void cal ( int sel )
{
    switch (sel) {
        case 1:                  /* 计算球体积 */
            printf("球体积为: %.2f\n",vol_ball());
            break;
        case 2:                  /* 计算圆柱体积 */
            printf("圆柱体积为: %.2f\n",vol_cylind( ));
            break;
        case 3:                  /* 计算圆锥体积 */
            printf("圆锥体积为: %.2f\n",vol_cone( ));
            break;
    }
}
```

```
/* 计算圆柱体积  $V=PI*r*r*h$  */
double vol_cylind( )
{
    double h, r;

    printf("请输入圆柱的底圆半径和高: ");
    scanf("%lf%lf", &r, &h);
    return(PI*r*r*h);
}
```

源程序

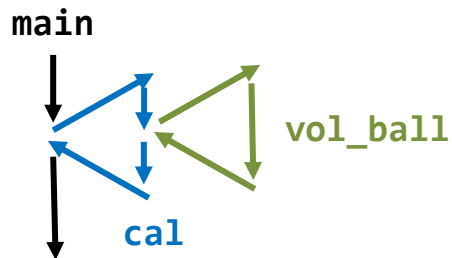
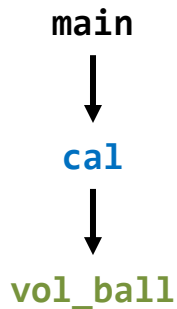
```
double  vol_ball(void);           /* 函数声明 */
double  vol_cylind(void);
double  vol_cone(void );
/* 常用圆形体体积计算器的主控函数 */
void cal ( int sel )
{
    switch (sel) {
        case 1:          /* 计算球体积 */
            printf("球体积为: %.2f\n",vol_ball());
            break;
        case 2:          /* 计算圆柱体积 */
            printf("圆柱体积为: %.2f\n",vol_cylind( ));
            break;
        case 3:          /* 计算圆锥体积 */
            printf("圆锥体积为: %.2f\n",vol_cone( ));
            break;
    }
}
```

```
/* 计算圆锥体积  $V=h/3*PI*r*r$  */
double vol_cone( )
{
    double h, r;

    printf("请输入圆锥的底圆半径和高: ");
    scanf("%lf%lf", &r,&h);
    return(PI*r*r*h/3.0);
}
```

嵌套调用

```
int main(void)
{
    .....
    cal(sel);
    .....
}
void cal(int sel)
{
    .....
    vol_ball();
    .....
}
double vol_ball()
{
    .....
}
```



结构化程序设计

□ 自顶向下，逐步求精，函数实现

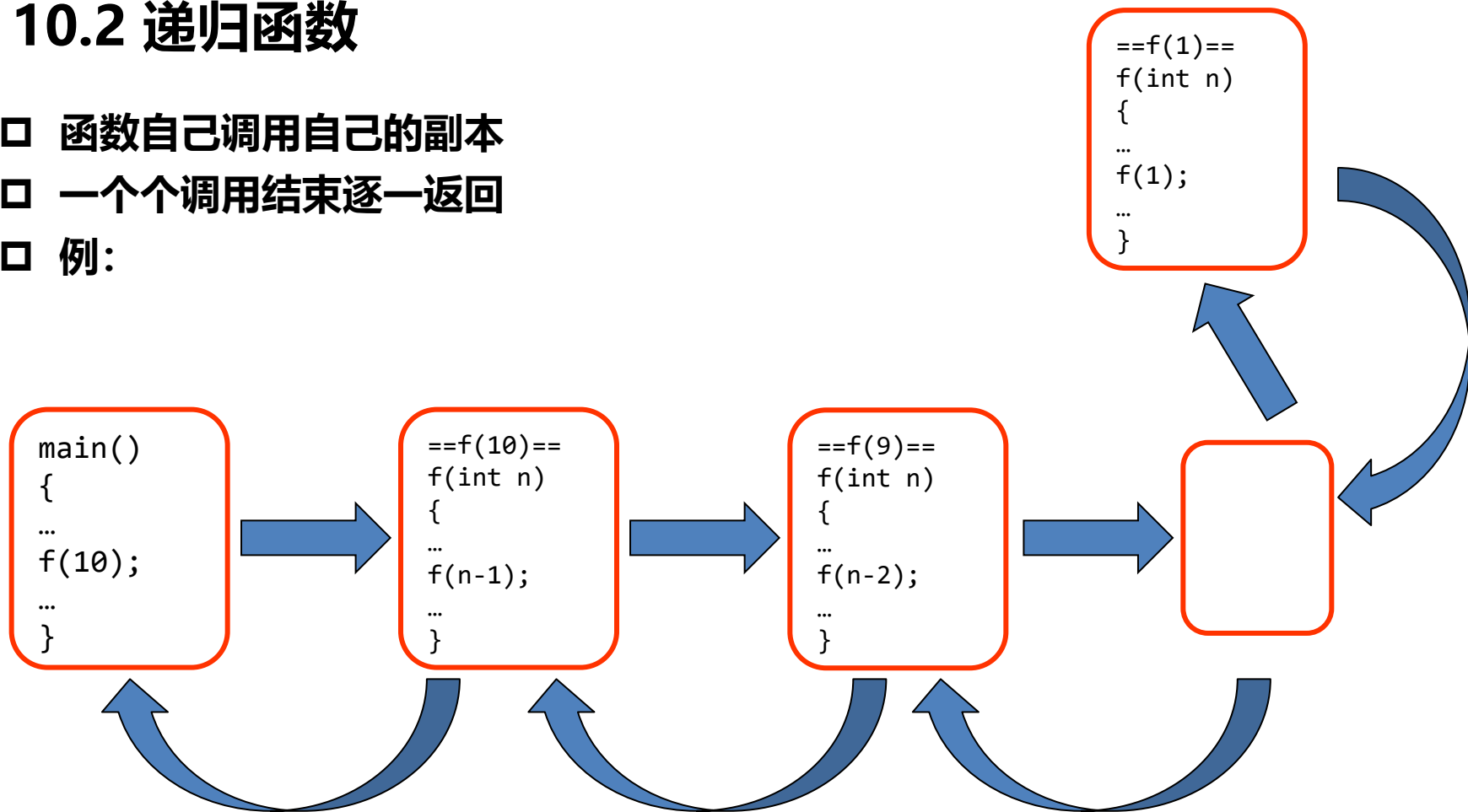
- **自顶向下**：程序设计时，应先考虑总体步骤，后考虑步骤的细节；先考虑全局目标，后考虑局部目标。先从最上层总目标开始设计，逐步使问题具体化。不要一开始就追求众多的细节
- **逐步求精**：对于复杂的问题，其中大的操作步骤再将其分解为一些子步骤的序列，逐步明晰实现过程
- **函数实现**：通过逐步求精，把程序要解决的全局目标分解为局部目标，再进一步分解为具体的小目标，把最终的小目标用函数来实现。问题的逐步分解关系，构成了函数间的调用关系

函数设计时应注意的问题

- **限制函数的长度：**一个函数语句数不宜过多，既便于阅读、理解，也方便程序调试。若函数太长，可以考虑把函数进一步分解实现
- **避免函数功能间的重复：**对于在多处使用的同一个计算或操作过程，应当将其封装成一个独立的函数，以达到一处定义、多处使用的目的，以避免功能模块间的重复
- **减少全局变量的使用：**应采用定义局部变量作为函数的临时工作单元，使用参数和返回值作为函数与外部进行数据交换的方式。只有当确实需要多个函数共享的数据时，才定义其为全局变量

10.2 递归函数

- 函数自己调用自己的副本
- 一个个调用结束逐一返回
- 例:



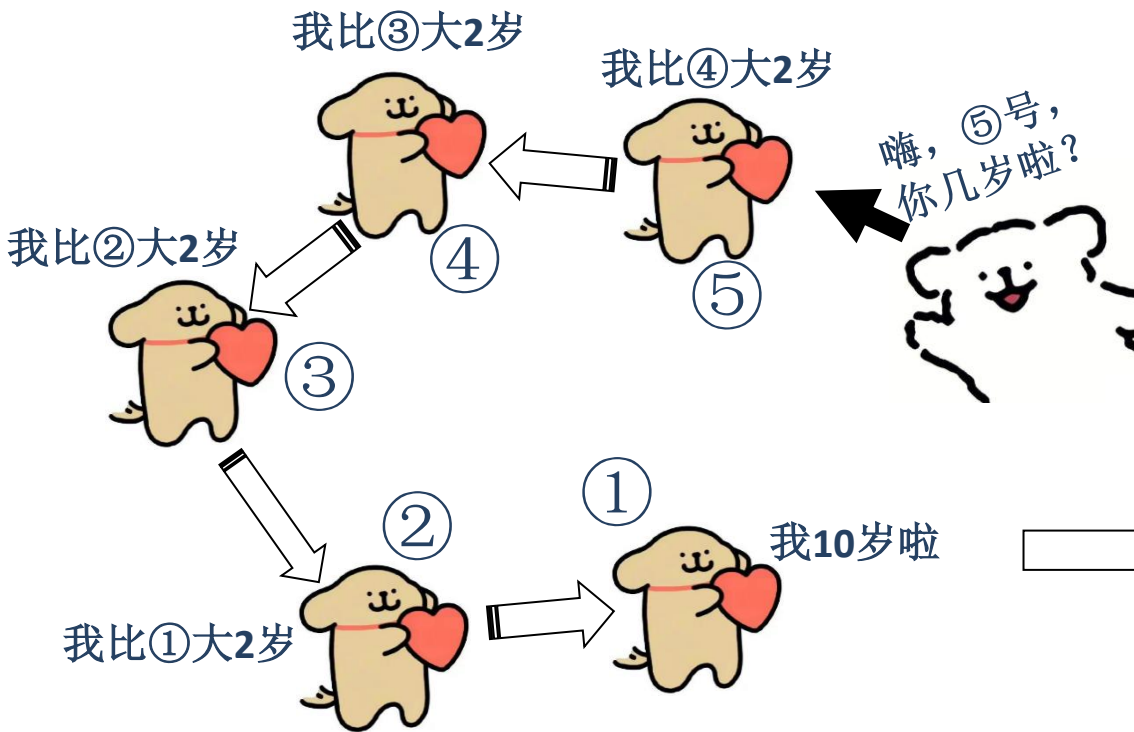
两种递归调用形式

函数直接递归调用	函数间接递归调用	
<pre>int f(int x) { int y; y = f(x); return y; }</pre>	<pre>int f(int x) { int y; y = g(x); return y; }</pre>	<pre>int g(int x) { int z; z = f(x); return z; }</pre>

10.2 递归函数

- 递归是指把一个**大问题**转化成**同样形式**但小一些的问题加以解决
- Recursion: occurs when a thing is defined in terms of itself or of its type
- **问题**: 有5个人坐在一起, 问第5个人多少岁? 他说比第4个人大2岁。问第4个人岁数, 他说比第3个人大2岁。问第3个人, 又说比第2个人大2岁。问第2个人, 说比第1个人大2岁。最后问第1个人, he 说是10岁。请问第5个人多大?

第5个人几岁



答案： $10+2+2+2+2=18$

$\text{age}(5) = \text{age}(4) + 2$
 $\text{age}(4) = \text{age}(3) + 2$
 $\text{age}(3) = \text{age}(2) + 2$
 $\text{age}(2) = \text{age}(1) + 2$
 $\text{age}(1) = 10$

⑤号到底几岁？

第5个人几岁

□ 用数学公式表述如下：

$$\text{age}(n) = \begin{cases} 10 & (n = 1) \\ \text{age}(n - 1) + 2 & (n > 1) \end{cases}$$

```
/* 可以用一个函数来描述上述递归过程 */  
int calcAge(int id)  
{  
    int result;  
    if(id == 1){  
        result = 10;  
    }  
    else{  
        result = calcAge(id - 1) + 2;  
    }  
    return result;  
}
```

递归出口

递归式

Fibonacci数列

$\text{fib}(n) = 0$

$n = 0$

$\text{fib}(n) = 1$

$n = 1, 2$

$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$

$n \geq 3$

```
int fib(int n)
{
    switch(n){
        case 0: return 0;
        case 1:
        case 2: return 1;
    }
    return fib(n - 1) + fib(n - 2);
}
```

递归出口

递归式

例10-2 计算整数阶乘

- 问题：计算整数 n 的阶乘 $n!$
- 方法1(递推法、循环)： $n! = 1*2*3*.....*(n-1)*n$

```
int factorial(int n)
{
    int product, i;
    product = 1;
    for(i = 1; i <= n; i++){
        product *= i;
    }
    return product;
}
```

例10-2 计算整数阶乘

□ 方法2(递归法): $n! = \begin{cases} 1 & (n = 0 \text{ or } 1) \\ n*(n-1)! & (n > 0) \end{cases}$

```
int factorial(int n)
{
    int result;
    if(n == 1 || n == 0)
        result = 1;
    else
        result = n * factorial(n - 1);
    return result;
}
```

递归出口

递归式

递归调用过程-以阶乘为例

```
int factorial(int n)
{
    int result;
    if(n == 1 || n == 0)
        result = 1;
    else
        result = n * factorial(n - 1);
    return result;
}
```

```
int main()
{
    int result = factorial(3);
    printf("%d\n", result);
    return 0;
}
```

系统存储区

操作系统（如 windows）、语言系统（如 Visual C++）

程序区（C 程序代码）

如主函数、函数 `complex_add()`、函数 `complex_prod()` 等

用户存储区

数据区

静态存储区

全局变量

如 `result_real`, `result_imag`

静态局部变量

动态存储区

（如自动变量）

`main()` 变量区:

`real1`, `imag1`, `real2`, `imag2`

`complex_add()` 变量区:

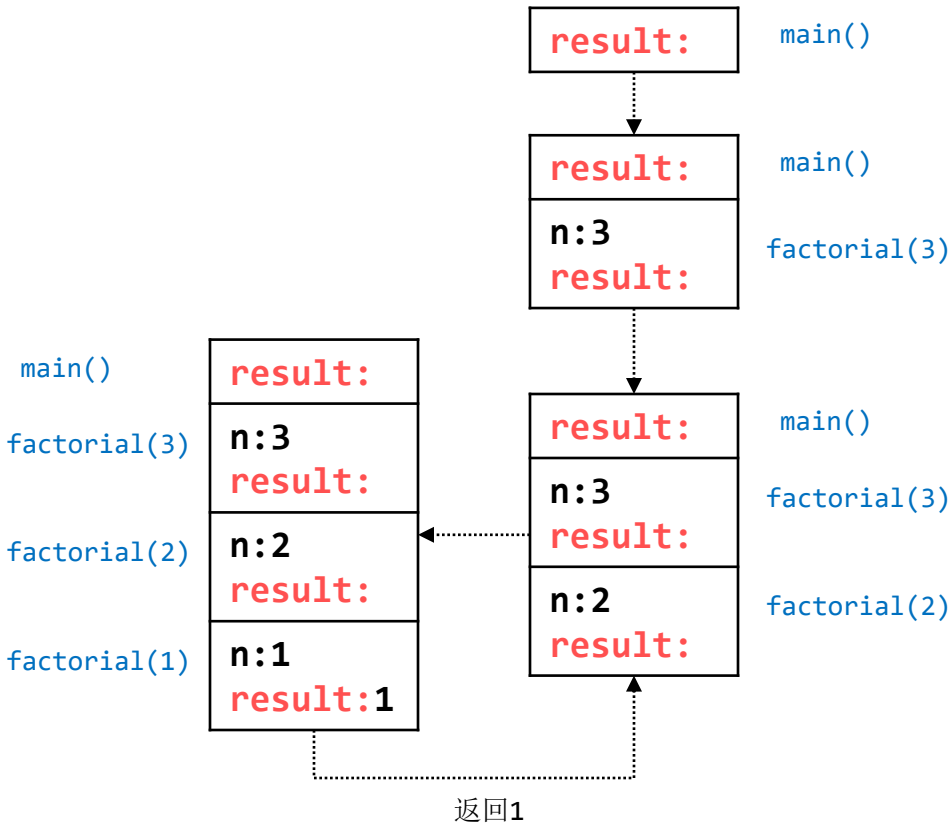
`real1`, `imag1`, `real2`, `imag2`

.....

递归调用过程-以阶乘为例

```
int factorial(int n)
{
    int result;
    if(n == 1 || n == 0)
        result = 1;
    else
        result = n * factorial(n - 1);
    return result;
}
```

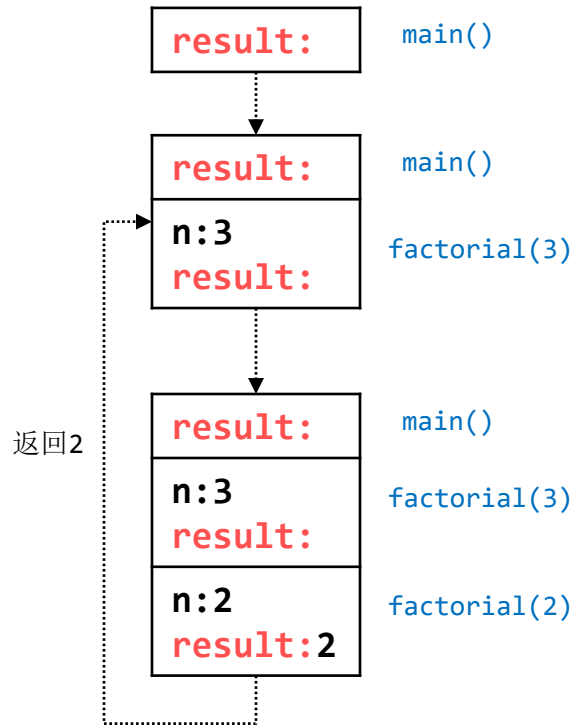
```
int main()
{
    int result = factorial(3);
    printf("%d\n", result);
    return 0;
}
```



递归调用过程-以阶乘为例

```
int factorial(int n)
{
    int result;
    if(n == 1 || n == 0)
        result = 1;
    else
        result = n * factorial(n - 1);
    return result;
}
```

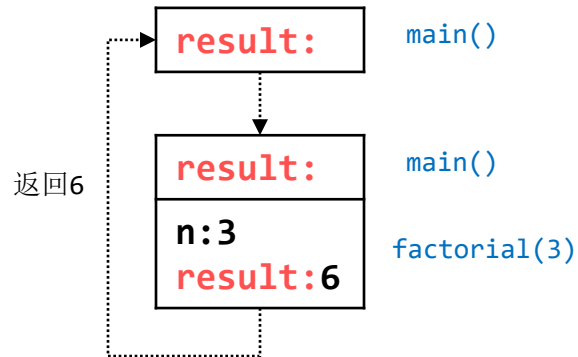
```
int main()
{
    int result = factorial(3);
    printf("%d\n", result);
    return 0;
}
```



递归调用过程-以阶乘为例

```
int factorial(int n)
{
    int result;
    if(n == 1 || n == 0)
        result = 1;
    else
        result = n * factorial(n - 1);
    return result;
}
```

```
int main()
{
    int result = factorial(3);
    printf("%d\n", result);
    return 0;
}
```



递归调用过程-以阶乘为例

```
int factorial(int n)
{
    int result;
    if(n == 1 || n == 0)
        result = 1;
    else
        result = n * factorial(n - 1);
    return result;
}
```

```
int main()
{
    int result = factorial(3);
    printf("%d\n", result);
    return 0;
}
```

result:6

main()

- 调用递归函数：调用另一个有着相同名字和相同代码的函数
- 每次调用函数时分配参数和局部变量的存储空间，退出函数时释放
- 随着递归函数的层层深入，存储空间的一端逐渐增加，然后随着函数调用的层层返回，存储空间的这一端又逐渐缩短
- 递归存在着可用堆栈空间耗尽的危险

递归函数正确性

□ 怎么证明递归函数是正确的呢？

- factorial(n)的正确性依赖于factorial(n-1)的正确性，依此类推，要证明factorial(1)的正确性，只要证明factorial(0)的正确性
- factorial(0)是正确的，调用它去完成另一个函数调用factorial(1)也应该是正确的，这种"相信"称为**Leap of Faith**
- 递归函数是正确的，只需证明两点：**Base Case正确，递推关系正确**
- 写递归函数一定要记得写Base Case，否则即使递推关系正确，整个函数也不正确

```
int factorial(int n)
{
    int result = n * factorial(n - 1);
    return result;
}
```

这个函数会永远调用下去，直到操作系统为程序预留的栈空间耗尽程序崩溃为止，这称为无穷递归

递归问题的说明

- 对于某一小范围内的问题，使用递归会带来简单、优雅的解。对于大多数问题，它所带来的解将会是极其复杂的。因此要**有选择地使用递归**
- 递归存在着可用堆栈空间过度使用的危险，这会导致严重的错误。在**安全相关**系统中强制规定：不能使用递归函数调用

10.2.3 递归程序设计

□ 用递归实现的问题，满足两个条件：

□ 问题可以逐步简化成自身较简单的形式（递归式）

$$n! = n * (n - 1)!$$

$$\sum_{i=1}^n i = n + \sum_{i=1}^{n-1} i$$

□ 递归最终能结束（递归出口）

□ 两个条件缺一不可

10

- 例10-3 定义函数gcd(m, n), 用递归求正整数m和n的最大公约数
 - 辗转相除法 (欧几里得算法)
 1. $r = m \% n$
 2. 若r为0, 则返回n的值; 否则转第(3)步
 3. $m = n, n = r$, 返回第(1)步
 - 递归实现的两个关键点:
 - 递归出口: n 当 $m\%n = 0$
 - 递归式子: 递归调用gcd(n, m%n) 当 $m\%n \neq 0$

例10-3 源程序

```
int gcd(int m, int n)
{
    if(m % n == 0){                /* 递归出口 */
        return n;
    }else{
        return gcd(n, m % n);    /* 递归调用 */
    }
}
```

10.2.3 递归程序设计

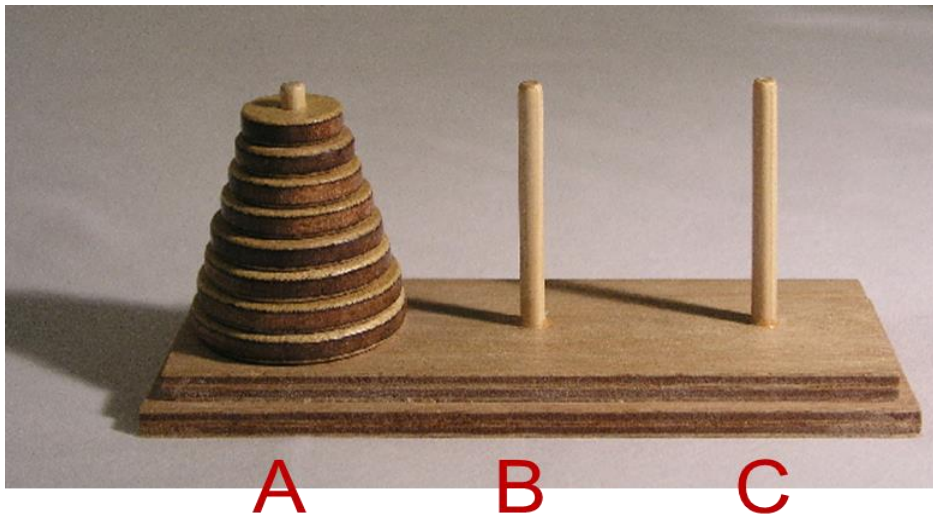
- 例10-4 编写递归函数reverse(int n)实现将正整数n逆序输出
- 分析：
 - 将正整数n逆序输出可以用循环实现，且循环次数与n的位数有关
 - 递归实现正整数逆序输出也需要用位数作为控制点。归纳递归实现的两个关键点如下：
 - 递归出口：直接输出n，如果 $n \leq 9$ ，即n为1位数
 - 递归式子：输出个位数 $n \% 10$ ，再递归调用reverse($n / 10$) 输出前n-1位，即n为多位数

例10-4 源程序

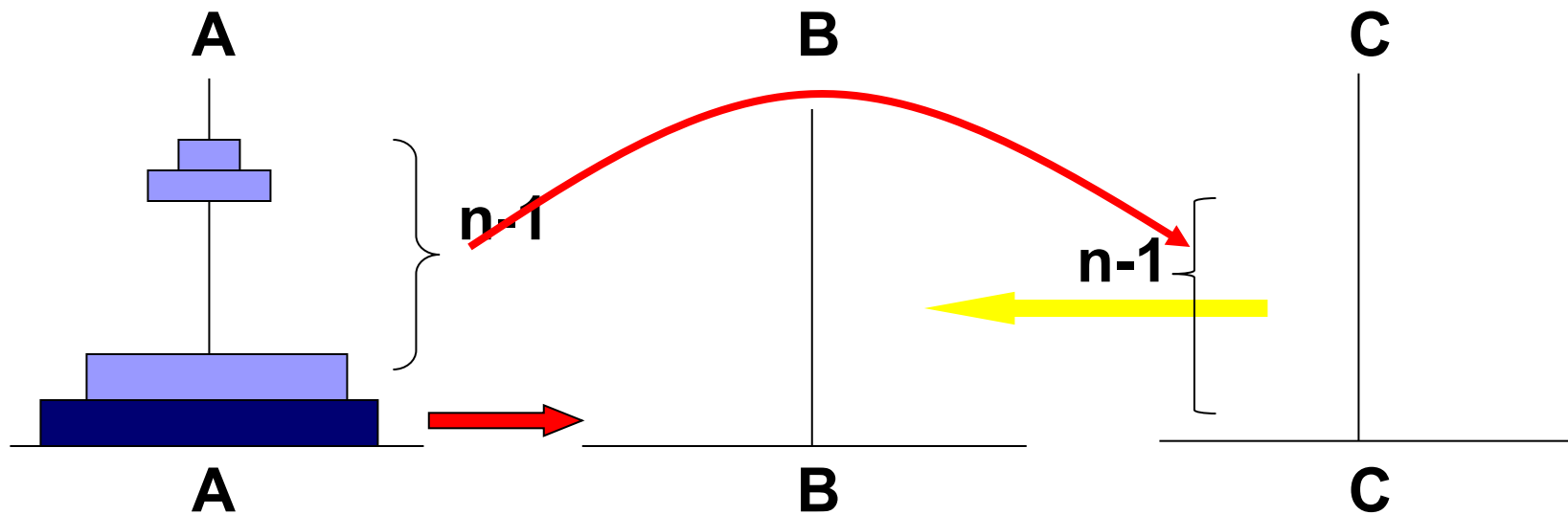
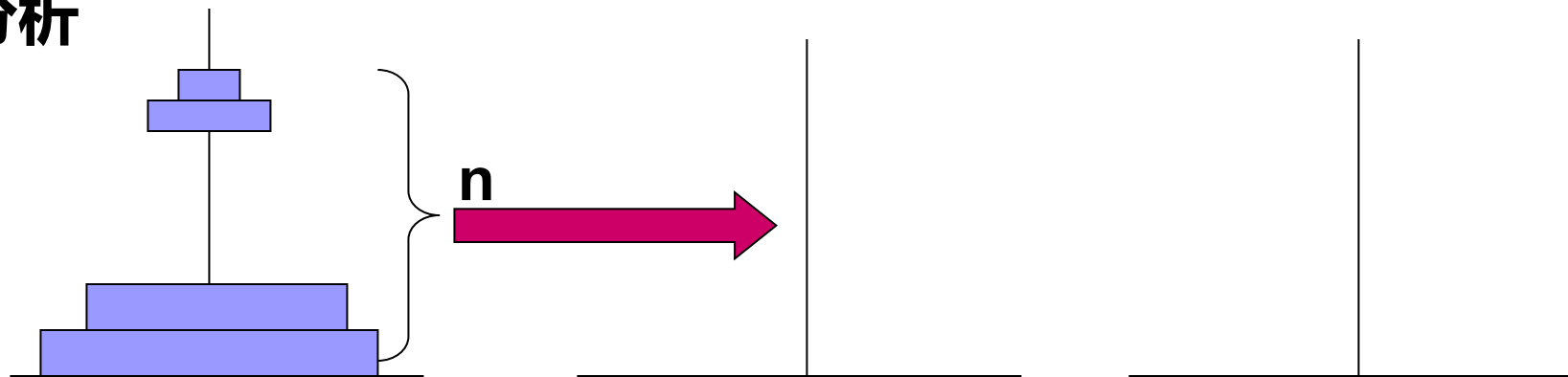
```
void reverse(int num)
{
    if (num<=9){
        printf ("%d",num);           /* 递归出口 */
    }else{
        printf ("%d",num%10);
        reverse (num/10);           /* 递归调用 */
    }
}
```

例10-5 汉诺塔问题

- 汉诺塔问题是源于印度一个古老传说的益智玩具。大梵天创造世界的时候做了三根金刚石柱子，在一根柱子上从下往上按照大小顺序摞着64片黄金圆盘。大梵天命令婆罗门把圆盘从下面开始按大小顺序重新摆放到另一根柱子上
- **规则：** 在小圆盘上不能放大圆盘，在三根柱子之间一次只能移动一个圆盘



分析



分析

□ **算法理解：** 要将A上的n个盘子按照要求移动到B上，我们可以：

- 先将A上的n-1个盘子移动到C上
- 再将A上的剩余的最大的盘子移动到B上
- 然后将C上的所有盘子移动到B上

□ **伪代码：**

```
void hanoi(n, A, B, C) /* A上的n个盘子按照要求移动到B上*/  
{  
    if(n == 1) move(A, B);  
    else{  
        hanoi(n-1, A, C, B);  
        move(A, B);  
        hanoi(n-1, C, B, A);  
    }  
}
```

汉诺塔源程序

```
#include <stdio.h>
void hanio(int n, char a, char b, char c);
int main(void)
{
    int n;
    printf("input the number of disk: " );
    scanf("%d", &n);
    printf("the steps for %d disks are:\n", n);
    hanio(n, 'a', 'b', 'c');
    return 0;
}
/* 搬运n个盘, 从a到b, c为中间过渡 */
void hanio(int n, char a, char b, char c)
{
    if(n == 1)
        printf("%c-->%c\n", a, b);
    else{
        hanio(n-1, a, c, b);
        printf("%c-->%c\n", a, b);
        hanio(n-1, c, b, a);
    }
}
```

```
/* 函数声明 */
```

```
input the number of disk: 3
the steps for 3 disks are:
a-->b
a-->c
b-->c
a-->b
c-->a
c-->b
a-->b
```

3 disks汉诺塔

```
input the number of disk: 3
the steps for 3 disks are:
a-->b
a-->c
b-->c
a-->b
c-->a
c-->b
a-->b
```

开始

动态展示

结束

第1步: a-->b

第2步: a-->c

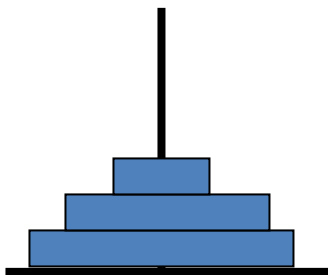
第3步: b-->c

第4步: a-->b

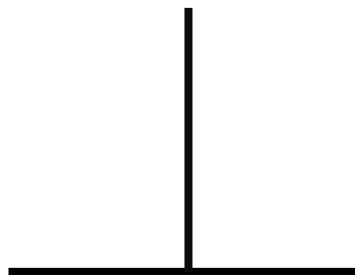
第5步: c-->a

第6步: c-->b

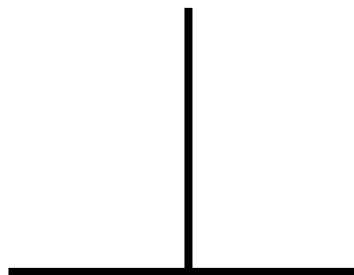
第7步: a-->b



A



B



C

10.2.3 递归程序设计

- 例10-6 老板有一袋金块，两名最优秀的雇员每人可以得到其中的一块，排名第一的得到最重的金块，排名第二的则得到袋子中最轻的金块
- 输入n（共n块， $2 \leq n \leq 100$ ）及n个整数，用分治法求出最重金块和最轻金块
 - 定义递归函数`max(int a[], int m, int n)`，在`a[m]~a[n]`中找出最大值
 - 递归出口：`a[m]`，当`m == n`，即a中只有1个元素
 - 递归式：将数组a分割为两部分，分别递归求最大值
 - `k = (m + n) / 2;`
 - `u = max(a, m, k);`
 - `v = max(a, k+1, n);`

例10-6 源程序

```
/* 分治法求a[m]~a[n]中最大值的递归函数 */
int max(int a[ ], int m, int n)
{
    int k, u, v;
    if (m == n) {
        return a[m];
    }
    k = (m + n) / 2;          /* 计算中间元素的下标k */
    u = max(a, m, k);        /* 在a[m]~a[k]中找出最大值 */
    v = max(a, k+1, n);      /* 在a[k+1]~a[n]中找出最大值 */

    return (u > v) ? u : v;  /* 返回u和v中较大的值 */
}
```

思考：整数划分问题

编程序求某一个正整数的所有划分数

如输入6，则输出：（要求输出不能重复）

$$6 = 5 + 1$$

$$6 = 4 + 2$$

$$6 = 4 + 1 + 1$$

$$6 = 3 + 3$$

$$6 = 3 + 2 + 1$$

$$6 = 3 + 1 + 1 + 1$$

$$6 = 2 + 2 + 2$$

$$6 = 2 + 2 + 1 + 1$$

$$6 = 2 + 1 + 1 + 1 + 1$$

$$6 = 1 + 1 + 1 + 1 + 1 + 1$$

编译预处理

□ C程序的编译过程

C源程序 → 编译预处理 → 编译 → 优化程序 → 汇编程序 → 链接程序 → 可执行文件

□ 编译预处理是C语言编译程序的组成部分，用于解释处理C语言源程序中的各种预处理指令

- 主要的预处理指令：文件包含(`#include`)，宏定义(`#define`)，条件编译
 - 在形式上都以"#"开头，不属于C语言中真正的语句
 - 增强了C语言的编程功能，改进C语言程序设计环境，提高编程效率
 - 由于`#define`等编译预处理指令不是C语句，不能被编译程序翻译，需要在真正编译之前作一个预处理，解释完成编译预处理指令，从而把预处理指令转换成相应的C程序段，最终成为由纯粹C语句构成的程序，经编译最后得到目标代码

宏定义

□ 例10-7 欧美国家长度使用英制单位，1英里=1609米，1英尺=34.48厘米，1英寸=2.54厘米。请编写程序转换单位

```
#include<stdio.h>
#define Mile_to_meter 1609          /* 1英里=1609米 */
#define Foot_to_centimeter 30.48    /* 1英尺=30.48厘米 */
#define Inch_to_centimeter 2.54     /* 1英寸=2.54厘米 */
int main(void)
{
    float foot, inch, mile;          /* 定义英里, 英尺, 英寸变量 */

    printf("Input mile, foot and inch:");
    scanf("%f%f%f", &mile, &foot, &inch); /* 分别输入英里, 英尺, 英寸 */
    printf("%f miles=%f meters\n", mile, mile * Mile_to_meter); /* 计算英里的米数 */
    printf("%f feet=%f centimeters\n", foot, foot * Foot_to_centimeter); /* 计算英尺的厘米数 */
    printf("%f inches=%f centimeters\n", inch, inch * Inch_to_centimeter); /* 计算英寸的厘米数 */

    return 0;
}
```

```
Input mile, foot and inch:1.2 3 5.1
1.200000 miles=1930.800049 meters
3.000000 feet=91.440000 centimeters
5.100000 inches=12.954000 centimeters
```


宏定义

#define 宏名 宏定义字符串

- 编译时，把程序中所有与宏名相同的字符串，用宏定义字符串替代

```
#define PI 3.14
```

```
#define attr_size 4
```

- 说明：

- 宏名一般用大写字母，以与变量名区别

- 宏定义不是C语句，后面不得跟分号

- 宏定义可以嵌套使用：

```
#define PI 3.14
```

```
#define S 2*PI*PI
```

- 宏定义可以写在程序的任何位置，它的作用范围从定义书写处到文件尾
- 可以通过"#undef"强制指定宏的结束范围

宏定义的作用范围

```
#define A "This is the first macro\n"
void f1()
{
    printf(A);
}
#define B "This is the second macro"
void f2()
{
    printf(B);
}
#undef B
int main(void)
{
    f1();
    f2();
    return 0;
}
```

B的有效范围

A的有效范围

带参数的宏定义

例10-8 简单的带参数宏定义

```
#include<stdio.h>
#define MAX(a, b) a>b?a:b
#define SQR(x) x*x
int main(void)
{
    int x, y;
    scanf("%d%d", &x, &y);
    x = MAX(x, y);
    y = SQR(x);
    printf("%d %d\n", x, y);
    return 0;
}
```

这样定义宏是否有问题？

假如程序中包含这样的语句：

```
x = 1, y = 2;
3*MAX(x,y)
SQR(x+y)
```

替换之后：

```
3*x>y?x:y
x+y*x+y
```

怎么避免这种情况？

宏定义示例

- ❑ 定义宏LOWCASE，判断字符c是否为小写字母

```
#define LOWCASE(c) (((c) >= 'a') && ((c) <= 'z'))
```

- ❑ 定义宏CTOD将数字字符('0'~'9')转换为相应的十进制整数，-1表示出错

```
#define CTOD(c) (((('c') >= '0') && ((c) <= '9')) ? c - '0' : -1)
```

练习

```
#define F(x) x - 2
#define D(x) x * F(x)

int main()
{
    printf("%d %d", D(3), D(D(3)));
    return 0;
}
```

宏定义示例

- 阅读带宏定义的程序，先全部替换好，最后再统一计算
- 不可一边替换一边计算，更不可以人为添加括号

```
#define F(x) x - 2  
#define D(x) x * F(x)
```

$D(3) = x * F(x)$

$= x * x - 2$

$= 3 * 3 - 2 = 7$

先用x替换展开

进一步对F(x)展开，这里不能加括号

最后把x=3代进去计算

$D(D(3)) = D(x * x - 2)$

$= x * x - 2 * F(x * x - 2)$

$= x * x - 2 * x * x - 2 - 2$

$= 3 * 3 - 2 * 3 * 3 - 2 - 2 = -13$

先用D(3)用x替换展开

拿展开后的参数对D进一步进行宏替换

拿展开后的参数对F进一步进行宏替换

最后把x=3代进去计算

练习

```
#define F1(var) printf("var = %d\n", var)
#define F0(var) F1(var*var)
```

3+4*3+4

```
int main()
{
    F0(3+4);
    return 0;
}
```

程序的输出是什么?

是 `var = 19`

还是 `3+4*3+4 = 19`

为什么字符串中的var不做替换?

如何让"var = %d\n"中的var也成为宏名?

改为"`#var`" = %d\n"

自行查阅资料了解宏定义中#和##的用法

文件包含

#include <需包含的文件名>

#include "需包含的文件名"

系统目录

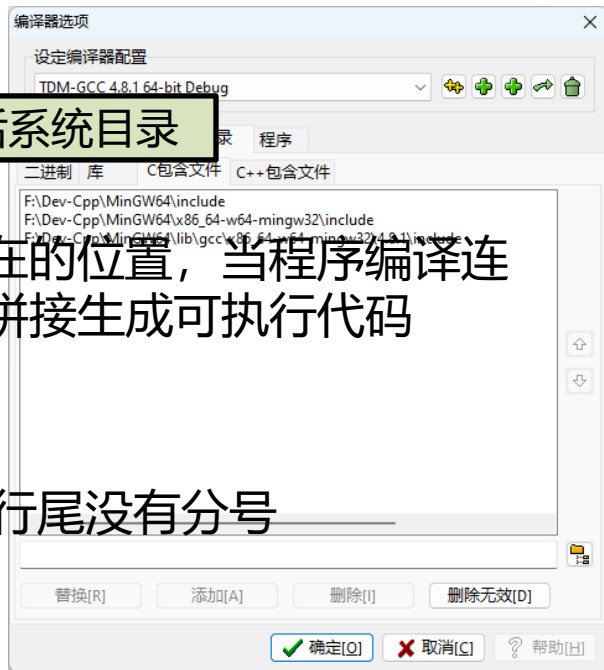
先工作目录，后系统目录

□ 作用：

- 把指定的文件模块内容插入到#include所在的位置，当程序编译连接时，系统会把所有#include指定的文件拼接生成可执行代码

□ 注意：

- 编译预处理命令，以#开头
- 在程序编译时起作用，不是真正的C语句，行尾没有分号



文件包含

例10-9 将例10-7中长度转换的宏，定义成头文件length.h，并写出主函数文件

头文件length.h源程序

```
#define Mile_to_meter 1609
#define Foot_to_centimeter 30.48
#define Inch_to_centimeter 2.54
```

主函数文件prog.c源程序

```
#include<stdio.h>
#include "length.h"      /* 包含自定义头文件 */
int main(void)
{
    float foot, inch, mile;          /* 定义英里, 英尺, 英寸变量 */
    printf("Input mile,foot and inch:");
    scanf("%f%f%f", &mile, &foot, &inch);          /* 分别输入英里, 英尺, 英寸 */
    printf("%f miles=%f meters\n", mile, mile * Mile_to_meter);          /* 计算英里的米数 */
    printf("%f feet=%f centimeters\n", foot, foot * Foot_to_centimeter);          /* 计算英尺的厘米数 */
    printf("%f inches=%f centimeters\n", inch, inch * Inch_to_centimeter);          /* 计算英寸的厘米数 */
    return 0;
}
```

常用标准头文件

- `ctype.h` 字符处理
- `math.h` 与数学处理函数有关的说明与定义
- `stdio.h` 输入输出函数中使用的有关说明和定义
- `string.h` 字符串函数的有关说明和定义
- `stddef.h` 定义某些常用内容
- `stdlib.h` 杂项说明
- `time.h` 支持系统时间函数

头文件的作用

- **通过头文件来调用函数功能。**在很多场合，源代码不便（或不准）向用户公布，只要向用户提供头文件和二进制的库即可。用户只需要按照头文件中的接口声明来调用库功能，而不必关心接口是怎么实现的。编译器会从库中提取相应的代码。例如C语言标准库不提供源代码，只有二进制代码。
- **头文件能加强类型安全检查。**如果某个接口被实现或被使用时，其方式与头文件中的声明不一致，编译器就会指出错误，这一简单的规则能大大减轻程序员调试、改错的负担。

如何写头文件

□ 头文件由三部分内容组成：

- 头文件开头处的版权和版本声明

- 预处理块

- 函数、结构和枚举类型声明、外部变量声明、具名常量定义、typedef和宏等

□ 头文件应该只用于声明，而**不应该包含或生成占据存储空间的变量或函数的定义**

条件编译

- 程序开发过程一般包含开发、调试、发行等过程。其中调试过程中需要一些特殊的代码用于记录日志、打印调试信息等，而发行版不需要这些额外的代码。如何只维护一个项目，同时方便的进行切换？

```
#define DEBUG /*此时#ifdef DEBUG为真*/  
//#define DEBUG //此时为假  
int main()  
{  
    #ifdef DEBUG  
        代码段A;  
    #else  
        代码段B;  
    #endif  
    代码段C;  
    return 0;  
}
```

- 这样可以实现debug功能，每次要输出调试信息前，只需要#ifdef DEBUG判断一次。
- 条件编译似乎可以用注释来取代，但应当注意到C语言不支持嵌套的注释（假如DEBUG和RELEASE具有共同的部分代码），因此条件编译要更灵活。
- 建议：代码段不应该被“注释掉”。当源代码不需要被编译时，应该使用条件编译来完成。

条件编译例子

- 假如现在要开发一个C语言程序，让它输出红色的文字，并且要求跨平台，在Windows和Linux下都能运行。这个程序的难点在于，不同平台下控制文字颜色的代码不一样，我们必须能够识别出不同的平台。Windows有专有的宏 `_WIN32`，Linux有专有的宏 `__linux__`，以现有的知识，我们很容易就想到了 `if else`:

```
int main(){
    if(_WIN32){
        system("color 0c");
        printf("https://nesa.zju.edu.cn\n");
    }
    else if(__linux__){
        printf("\033[22;31mhttps://nesa.zju.edu.cn\n\033[22;30m");
    }
    else{
        printf("https://nesa.zju.edu.cn\n");
    }
    return 0;
}
```

但这段代码是错误的，在Windows下提示 `__linux__` 是未定义的标识符，在Linux下提示 `_Win32` 是未定义的标识符

条件编译例子

□ 对上面的代码改进如下：

```
1.  int main(){
2.      #if _WIN32
3.          system("color 0c");
4.          printf("https://nesa.zju.edu.cn\n");
5.      #elif __linux__
6.          printf("\033[22;31mhttps://nesa.zju.edu.cn\n\033[22;30m");
7.      #else
8.          printf("https://nesa.zju.edu.cn\n");
9.      #endif
10.
11.     return 0;
12. }
```

□ #if, #elif, #else和#endif都是预处理命令，整段代码的意思是：如果宏 `_WIN32` 的值为真，就保留第3、4行代码，删除第6、8行代码；如果宏 `__linux__` 的值为真，就保留第6行代码；如果所有的宏都为假，就保留第8行代码

#ifdef和#ifndef

#ifdef的用法

```
#ifdef 宏名
    程序段1
#else
    程序段2
#endif
```

如果宏名被定义过，则对程序段1进行编译，否则对程序段2进行编译

#ifndef的用法

```
#ifndef 宏名
    程序段1
#else
    程序段2
#endif
```

如果宏名未被定义过，则对程序段1进行编译，否则对程序段2进行编译

#if和#ifdef、#ifndef的区别

#if后面跟的是“整型常量表达式”，而#ifdef和#ifndef后面跟的只能是一个宏名

```
#include<stdio.h>
#define NUM 10
int main(){
    #if NUM == 10 || NUM == 20
        printf("NUM:%d\n", NUM);
    #else
        printf("NUM Error\n");
    #endif

    return 0;
}
```

```
#include<stdio.h>
#define NUM1 10
#define NUM2 20
int main(){
    #if (defined NUM1 && defined NUM2)
        printf("NUM1:%d, NUM2:%d\n", NUM1, NUM2);
    #else
        printf("NUM Error\n");
    #endif

    return 0;
}
```

问题

```
// 头文件header.h
```

```
char school[] = "浙江大学";
```

```
//f1.c
```

```
#include "header.h"
```

```
.....
```

```
//f2.c
```

```
#include "header.h"
```

```
.....
```

```
//main.c
```

```
#include "header.h"
```

```
#include "f1.c"
```

```
#include "f2.c"
```

```
int main(){
```

```
.....
```

```
}
```

link-time error: "multiple definition of..."

避免头文件多重包含(#define保护)

□ 头文件多重包含问题如何避免？

- 所有头文件都应该使用#define防止头文件被多重包含

```
#ifndef _HEADERNAME_H
#define _HEADERNAME_H
...//(头文件内容)
#endif
```

□ 使用头文件的注意点：

- 虽然函数、变量的声明都可以重复，所以同一个声明出现多次也不会影响程序的运行，但它会增加编译时间，**重复引用头文件会浪费编译时间**
- 当头文件中包含结构的定义、枚举定义等一些定义时，这些定义是不可以重复的，**必须通过一定措施防止重复引用**

总结-编译预处理指令

指令	说明
#	空指令，无任何效果
#include	包含一个源代码文件
#define	定义宏
#undef	取消已定义的宏
#if	如果给定条件为真，则编译下面代码
#ifdef	如果宏已经定义，则编译下面代码
#ifndef	如果宏没有定义，则编译下面代码
#elif	如果前面的 #if 给定条件不为真，当前条件为真，则编译下面代码
#endif	结束一个 #if.....#else 条件编译块

程序文件模块

- 结构化程序设计是编写出具有良好程序结构的有效方法
- **一个C程序由一组函数构成，但是**
 - 如果程序规模大，多人合作开发，每个人编写的代码会保存在自己的.c文件，这些文件代码一起组成一个大程序
 - 有时为避免一个文件过长，也会把程序代码分开保存为多个文件
- **结论：一个C大程序由多个程序文件组成，每个文件又包含若干个函数**
- **包含main函数的模块叫主模块，有且只有一个**

文件模块间的通信

□ 变量

- 外部变量

- 静态全局变量

□ 函数

- 外部函数

- 静态函数

外部变量

- 全局变量只能在某个模块中定义一次，如果其他模块要使用该全局变量，需要通过外部变量的声明，声明格式为 **extern 类型名 变量名表;**
- 如果在每个文件模块中都定义同一个全局变量，模块在单独编译时不会发生错误，一旦把各模块连接在一起时，就会产生对同一个全局变量名多次定义的错误
- 反之，不经声明而直接使用全局变量，程序编译时会出现“变量未定义”的错误

File1.c

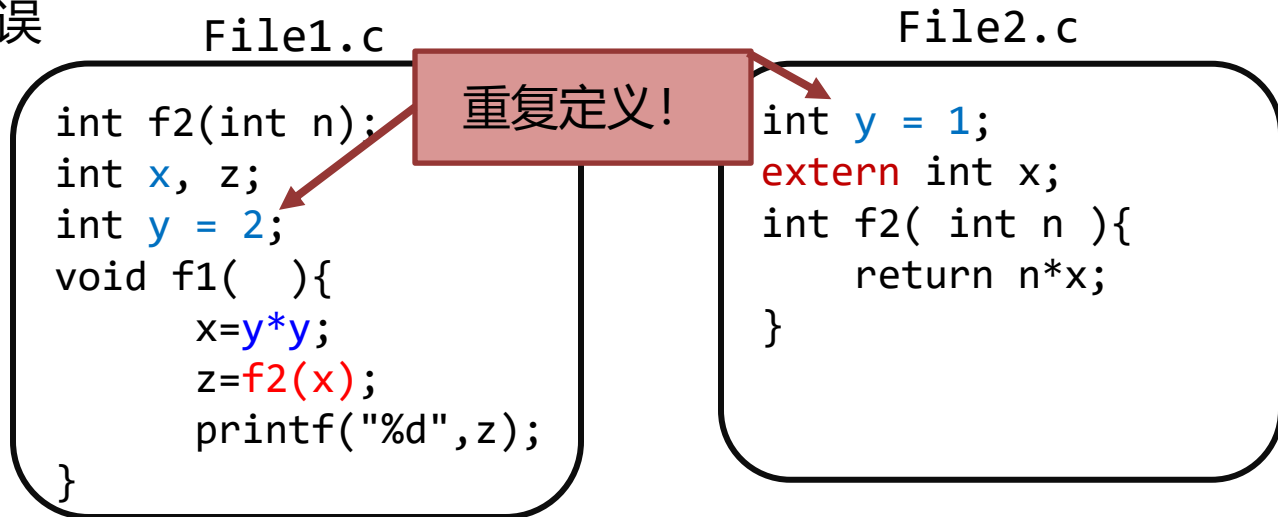
```
int f2(int n);  
int x, z;  
int y = 2;  
void f1( ) {  
    x=y*y;  
    z=f2(x);  
    printf("%d",z);  
}
```

File2.c

```
int y = 1;  
extern int x;  
int f2( int n ) {  
    return n*x;  
}
```

外部变量

- 全局变量只能在某个模块中定义一次，如果其他模块要使用该全局变量，需要通过外部变量的声明，声明格式为 **extern 类型名 变量名表;**
- 如果在每个文件模块中都定义同一个全局变量，模块在单独编译时不会发生错误，一旦把各模块连接在一起时，就会产生对同一个全局变量名多次定义的错误
- 反之，不经声明而直接使用全局变量，程序编译时会出现“变量未定义”的错误



外部变量

- 全局变量只能在某个模块中定义一次，如果其他模块要使用该全局变量，需要通过外部变量的声明，声明格式为 **extern 类型名 变量名表;**
- 如果在每个文件模块中都定义同一个全局变量，模块在单独编译时不会发生错误，一旦把各模块连接在一起时，就会产生对同一个全局变量名多次定义的错误
- 反之，不经声明而直接使用全局变量，程序编译时会出现“变量未定义”的错误

File1.c

```
int f2(int n);  
int x, z;  
void f1( ) {  
    x=y*y;  
    z=f2(x);  
    printf("%d",z);  
}
```

File2.c

```
int y = 1;  
extern int x;  
int f2( int n ) {  
    return n*x;  
}
```

外部变量

- 全局变量只能在某个模块中定义一次，如果其他模块要使用该全局变量，需要通过外部变量的声明，声明格式为 **extern 类型名 变量名表;**
- 如果在每个文件模块中都定义同一个全局变量，模块在单独编译时不会发生错误，一旦把各模块连接在一起时，就会产生对同一个全局变量名多次定义的错误
- 反之，不经声明而直接使用全局变量，程序编译时会出现“变量未定义”的错误

File1.c

```
int f2(int n);  
int x, z;  
void f1( ) {  
    x=y*y;  
    z=f2(x);  
    printf("%d", z);  
}
```

File2.c

```
int y = 1;  
extern int x;  
int f2( int n ) {  
    return n*x;  
}
```

变量未定义!

外部变量

- 全局变量只能在某个模块中定义一次，如果其他模块要使用该全局变量，需要通过外部变量的声明，声明格式为 **extern 类型名 变量名表;**
- 如果在每个文件模块中都定义同一个全局变量，模块在单独编译时不会发生错误，一旦把各模块连接在一起时，就会产生对同一个全局变量名多次定义的错误
- 反之，不经声明而直接使用全局变量，程序编译时会出现“变量未定义”的错误

File1.c

```
int f2(int n);  
int x, z;  
extern int y;  
void f1( ) {  
    x=y*y;  
    z=f2(x);  
    printf("%d",z);  
}
```

正确声明
外部全局变量

File2.c

```
int y = 1;  
extern int x;  
int f2( int n ) {  
    return n*x;  
}
```

限制其他文件访问：静态全局变量

- 当一个大程序由多人合作完成时，每个程序员可能都会定义一些自己使用的全局变量
- 为避免自己定义的全局变量影响其他人编写的模块，即所谓的全局变量副作用，静态全局变量可以把变量的作用范围仅局限于当前的文件模块中
- 即使其他文件模块使用外部变量声明，也不能使用该变量
- 格式： `static 类型名 变量名`

外部函数与静态函数

□ 外部函数

- 如果要在一个模块中调用另一个模块中的函数时，就需要对函数进行外部声明，声明格式为

`extern 函数类型 函数名(参数表)`

□ 静态函数

- 把函数的使用范围限制在文件模块内，不使某程序员编写的自用函数影响其他程序员的程序，即使其他文件模块有同名的函数定义，相互间也没有任何关联，增加模块的独立性
- 方法：static关键字声明，如`static int f(int n);`

高质量的函数设计原则

- **好的函数名字**，描述函数所做的事情。如checkOrderInfo(), calMonthlyRevenues()
- **高内聚性**：尽量一个函数只实现一个简单功能
- **如果一个函数的代码行数很大（比如150以上），把它分成几个相互调用的小函数来完成，每个函数一般不要超过60行代码**
- **函数参数设计**：
 - 按照输入、修改、输出的顺序排列参数
 - 考虑对参数采用某种表示输入、修改、输出的命名规则
 - 使用所有的参数
 - 把状态或出错变量放在最后
 - 不要把函数的参数用作工作变量
 - 在接口中对参数的假定加以说明
 - 把函数的参数个数限制在大约7个以内

编码规范

□ 什么是高质量的程序

- **正确性**：语法正确、功能正确。使之可行

- **可读性**：通用的、必须的惯用语和模式使代码更加容易理解。使之优雅

- **可维护性**：程序应对变化的能力。使之优化

-

□ 编码规范很重要，特别是团队开发的时候

巧用typedef

- ❑ **不直接使用基本数据类型**，应该使用指示了大小和符号的typedef以代替基本数据类型
- ❑ 比如，《MISRA C-2008工业标准》建议为所有基本数值类型和字符类型使用如下的typedef。对于32位计算机来说，它们是：

```
typedef char          char_t;  
typedef signed char   int8_t;  
typedef signed short  int16_t;  
typedef signed int     int32_t;  
typedef signed long    int64_t;  
typedef unsigned char uint8_t;
```

```
typedef unsigned short uint16_t;  
typedef unsigned int   uint32_t;  
typedef unsigned long  uint64_t;  
typedef float           float32_t;  
typedef double          float64_t;  
typedef long double     float128_t;
```


其它编码规范

□ 变量、函数的命名符合编码规范

- Pascal命名规则：当变量名和函数名称是由两个或两个以上单词连结在一起，而构成的唯一识别字时，第一个单词首字母采用大写字母，后续单词的首字母亦采用大写字母，例如：FirstName, LastName

□ 小心使用全局变量

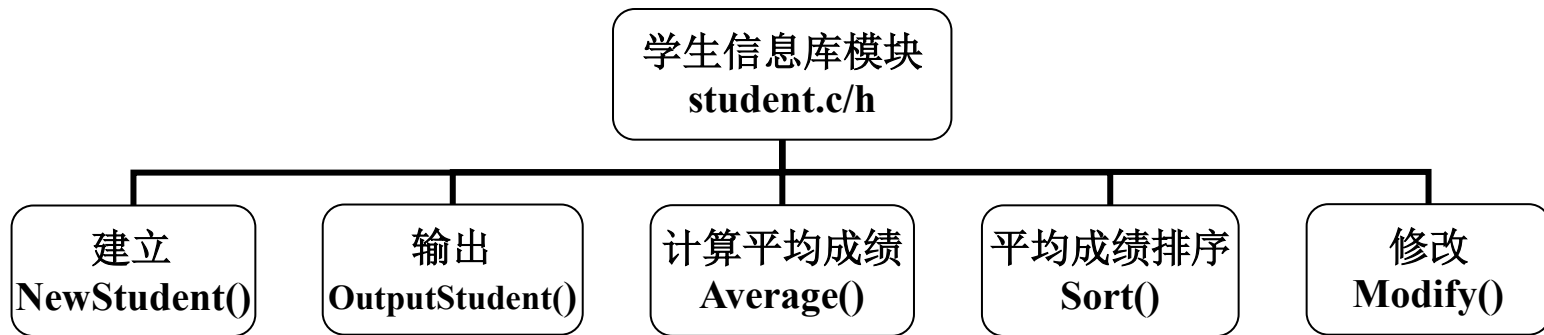
- 多线程代码中非常数的全局变量是禁止使用的。内建类型的全局变量是允许的，但慎用

□ 用访问器子程序来取代全局数据

- 把数据隐藏到模块里面。用static关键字来定义该数据，写出可以read、write和initialize的子程序来。要求外部模块通过子程序来访问数据，而不能直接操作它

10.4 大程序构成

- 例10-10(改编) 请实现一个学生信息库模块。该系统包含学生基本信息的建立和输出、计算学生平均成绩、按照学生的平均成绩排序以及修改学生的成绩等功能



- 如何创建项目进行多文件模块化编程请参考《Dev C++及其它编译器编程基础》

student.h

```
#ifndef __STUDENT_H__
#define __STUDENT_H__

typedef enum {NUM, NAME, COMPUTER, ENGLISH, MATH} AttrType; /*枚举可修改的字段*/
typedef union { /*修改的值*/
    int num;
    char name[10];
    int computer;
    int english;
    int math;
} AttrValue;

typedef struct{ /*学生信息结构体*/
    int num;
    char name[10];
    int computer, english, math;
    double average;
} Stu;
typedef Stu *StuPt;
```

student.h

```
typedef struct{           /*学生列表结构体*/
    StuPt head;
    int count;
    int capacity;
} StuList;
typedef StuList *StuListPt;

extern StuListPt Init();
extern void Close(StuListPt sl);
extern StuListPt NewStudent(StuListPt sl, int num, char *name, int computer, int english, int math);
extern void OutputStudent(StuListPt sl);
extern void Average(StuListPt sl);
extern void Modify(StuListPt sl, int num, AttrType t, AttrValue v);
extern void Sort(StuListPt sl);
static StuPt SearchStudent(StuListPt sl, int num);

#endif
```

头文件中声明的数据类型、函数可以被外部模块使用

static修饰的函数仅供模块内部使用，外部模块无法调用

student.c / Init()

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "student.h"

#define DEFAULT_CAPACITY 5 /*定义在student.c内部, 只能被本模块使用*/

StuListPt Init()
{
    StuListPt sl;
    sl = (StuListPt)malloc(sizeof(StuList));
    if(sl == NULL)
    {
        printf("Failed to initialize student list.\n");
        return NULL;
    }
    sl->count = 0;          /*学生个数*/
    sl->capacity = DEFAULT_CAPACITY; /*默认容量, 可扩展*/
    sl->head = (StuPt)malloc(sizeof(Stu) * sl->capacity);
    if(sl->head == NULL)
    {
        free(sl);
        printf("Failed to initialize student list.\n");
        return NULL;
    }
    return sl;
}
```

student.c / Close()

```
void Close(StuListPt s1)
{
    /*free函数不会递归回收资源，先回收内部动态存储，再回收外部动态存储*/
    free(s1->head);
    free(s1);
}
```

student.c / NewStudent()

```
StuListPt NewStudent(StuListPt sl, int num, char *name, int computer, int english, int math)
{
    if(num < 0 || strlen(name) >= 10 || computer < 0 || english < 0 || math < 0)
    {
        printf("Invalid value for input student info.\n");
        return sl;
    }

    if(SearchStudent(sl, num) != NULL)           /*如果此学生已在系统中*/
    {
        printf("Student %d already exists.\n", num);
        return sl;
    }

    if(sl->count == sl->capacity)                 /*超出了最大限度, 扩大存储空间*/
    {
        StuPt new_head = (StuPt)realloc(sl->head, sizeof(Stu) * sl->capacity * 2);
        if(new_head == NULL)
        {
            printf("Failed to insert new student.\n");
            return sl;
        }
        sl->head = new_head;
        sl->capacity *= 2;
    }
}
```

student.c / NewStudent() (续)

```
s1->head[s1->count].num = num;  
strcpy(s1->head[s1->count].name, name);  
s1->head[s1->count].computer = computer;  
s1->head[s1->count].english = english;  
s1->head[s1->count].math = math;
```

```
s1->count++;
```

```
return s1;
```

```
}
```


student.c / OutputStudent()

```
void OutputStudent(StuListPt sl)
{
    int i;

    printf("Total number of %d students.\n", sl->count);

    for(i = 0; i < sl->count; i++)
        printf(
            "Num: %d, Name: %s, Computer: %d, English: %d, Math: %d, Average: %.1f\n",
            sl->head[i].num,
            sl->head[i].name,
            sl->head[i].computer,
            sl->head[i].english,
            sl->head[i].math,
            sl->head[i].average
        );
}
```

11

[illegible]

student.c / Modify()

```
void Modify(StuListPt sl, int num, AttrType t, AttrValue v)
{
    int i;
    StuPt p = NULL;

    p = SearchStudent(sl, num); /*根据学号搜索对应的学生节点*/

    if(p != NULL)                /*根据字段类型修改对应的字段*/
        switch(t)
        {
            case NUM:           p->num = v.num; break;
            case NAME:          strcpy(p->name, v.name); break;
            case COMPUTER:      p->computer = v.computer; break;
            case ENGLISH:       p->english = v.english; break;
            case MATH:           p->math = v.math; break;
            default:             printf("Unrecognized input type.\n"); break;
        }
}
```

student.c / Sort()

```
void Sort(StuListPt sl)
{
    int i, j;
    StuPt head = sl->head;

    for(i = 1; i < sl->count; i++)
        for(j = 0; j < sl->count - i; j++)
        {
            if(head[j].average > head[j + 1].average)
            {
                Stu temp = head[j];
                head[j] = head[j + 1];
                head[j + 1] = temp;
            }
        }
}
```

/*冒泡排序, 根据平均成绩排序*/

student.c / SearchStudent()

```
static StuPt SearchStudent(StuListPt sl, int num) /*静态函数, 仅在内部模块使用*/
{
    int i;
    StuPt p = NULL;

    for(i = 0; i < sl->count; i++)
        if(sl->head[i].num == num)
        {
            p = sl->head + i;
            break;
        }

    return p;
}
```

main.c

```
#include <string.h>
#include "student.h"
int main()
{
    StuListPt h = Init();

    /*插入5条学生信息, 计算平均分*/
    h = NewStudent(h, 0, "Marry", 90, 95, 100);
    h = NewStudent(h, 4, "Jack", 60, 60, 60);
    h = NewStudent(h, 2, "Bob", 89, 93, 87);
    h = NewStudent(h, 3, "Ben", 91, 82, 100);
    h = NewStudent(h, 1, "Jessica", 100, 79, 92);
    Average(h);
    OutputStudent(h);

    /*排序后输出*/
    Sort(h);
    OutputStudent(h);
```

```
    /*修改Bob的名字为Bobby*/
    AttrType t = NAME;
    AttrValue v;
    strcpy(v.name, "Bobby");
    Modify(h, 2, t, v);
    OutputStudent(h);
```

```
    /*插入已存在的学生*/
    h = NewStudent(h, 0, "Marry", 90, 95, 100);
```

```
    /*超出最大容量后再插入2条信息*/
    h = NewStudent(h, 9, "Ross", 56, 67, 78);
    h = NewStudent(h, 5, "Rachael", 98, 95, 90);
    Average(h);
    OutputStudent(h);
```

```
    Close(h);
```

```
    return 0;
```

```
}
```

测试结果

```
Total number of 5 students.
Num: 0, Name: Marry, Computer: 90, English: 95, Math: 100, Average: 95.0
Num: 4, Name: Jack, Computer: 60, English: 60, Math: 60, Average: 60.0
Num: 2, Name: Bob, Computer: 89, English: 93, Math: 87, Average: 89.7
Num: 3, Name: Ben, Computer: 91, English: 82, Math: 100, Average: 91.0
Num: 1, Name: Jessica, Computer: 100, English: 79, Math: 92, Average: 90.3
Total number of 5 students.
Num: 4, Name: Jack, Computer: 60, English: 60, Math: 60, Average: 60.0
Num: 2, Name: Bob, Computer: 89, English: 93, Math: 87, Average: 89.7
Num: 1, Name: Jessica, Computer: 100, English: 79, Math: 92, Average: 90.3
Num: 3, Name: Ben, Computer: 91, English: 82, Math: 100, Average: 91.0
Num: 0, Name: Marry, Computer: 90, English: 95, Math: 100, Average: 95.0
Total number of 5 students.
Num: 4, Name: Jack, Computer: 60, English: 60, Math: 60, Average: 60.0
Num: 2, Name: Bobby, Computer: 89, English: 93, Math: 87, Average: 89.7
Num: 1, Name: Jessica, Computer: 100, English: 79, Math: 92, Average: 90.3
Num: 3, Name: Ben, Computer: 91, English: 82, Math: 100, Average: 91.0
Num: 0, Name: Marry, Computer: 90, English: 95, Math: 100, Average: 95.0
Student 0 already exists.
Total number of 7 students.
Num: 4, Name: Jack, Computer: 60, English: 60, Math: 60, Average: 60.0
Num: 2, Name: Bobby, Computer: 89, English: 93, Math: 87, Average: 89.7
Num: 1, Name: Jessica, Computer: 100, English: 79, Math: 92, Average: 90.3
Num: 3, Name: Ben, Computer: 91, English: 82, Math: 100, Average: 91.0
Num: 0, Name: Marry, Computer: 90, English: 95, Math: 100, Average: 95.0
Num: 9, Name: Ross, Computer: 56, English: 67, Math: 78, Average: 67.0
Num: 5, Name: Rachael, Computer: 98, English: 95, Math: 90, Average: 94.3

-----
Process exited after 0.03422 seconds with return value 0
请按任意键继续. . . |
```

本章小结

□ 多函数程序的组织结构

- 函数调用的层次结构
- 多文件模块实现：文件包含
- 合理运用变量在多文件模块、多函数间的关联
- 程序文件模块：变量与文件模块、函数与文件模块的关系

□ 递归函数

- 构成要素：递归式子（重点）与递归出口
- 运用递归函数解决特殊问题（如汉诺塔）

□ 编译预处理

- 文件包含
- 宏实质：编译预处理的替代
- 带参的宏——不是函数

谢 谢
